Informe de Cambios — ADS1198 Initialization Fix

Proyecto: ESP32 + ADS1198 | **Fecha:** Marzo 2026

Archivos modificados: `ADS119X.h` · `ADS119X.cpp` · `main.ino`

Objetivo: Corregir la secuencia de inicialización del ADS1198 conforme al datasheet SBAS471C (p. 65–66).

Índice

1. ADS119X.h — Header guard corregido
2. ADS119X.h — Nuevos miembros privados
3. ADS119X.cpp — Constructor: SPI eliminado, CS/RESET inicializados
4. ADS119X.cpp — begin(): secuencia tPOR + tRST
5. ADS119X.cpp — begin(): loop de 3 reintentos para validar ID
6. ADS119X.cpp — testSignal(): CMD_START añadido
7. ADS119X.cpp — testSignalDC(): CMD_START añadido
8. main.ino — Inicialización explícita de pines en setup()
9. main.ino — CMD_START explícito antes de RDATAC

C1 · `ADS119X.h` — Header guard corregido

Descripción: El include guard usaba `ADS1256_h` (nombre de un chip distinto de TI). Podía causar conflictos de inclusión múltiple.

```
- #ifndef ADS1256_h

- #define ADS1256_h

+ #ifndef ADS119X_h

+ #define ADS119X_h
```

[+] Pros	[!] Contras
Elimina riesgo de inclusión múltiple	Cambio puramente cosmético, sin impacto en runtime
Nombre coherente con el chip real (ADS1198)	—

C2 · `ADS119X.h` — Nuevos miembros privados para pines SPI

Descripción: El constructor recibía `clk`, `miso`, `mosi` pero los usaba directamente en `SPI.begin()` sin guardarlos. Al mover `SPI.begin()` a `begin()`, es necesario tenerlos accesibles como miembros.

```
  // pins
  byte _drdy_pin;
  byte _reset_pin;
  byte _cs_pin ;
+ byte _clk_pin;

+ byte _miso_pin;

+ byte _mosi_pin;
```

[+] Pros	[!] Contras
Desacopla configuración de pines del inicio SPI	Añade 3 bytes al objeto (irrelevante en ESP32)
Permite llamar `SPI.begin()` en el momento correcto	—

C3 · `ADS119X.cpp` — Constructor: SPI eliminado, CS/RESET inicializados

Descripción: El constructor llamaba `SPI.begin()` y `SPI.beginTransaction()` inmediatamente, antes de que el ADS1198 completara su power-on reset. Se reemplaza por la inicialización de niveles seguros en CS y RESET.

```
  // Constructor
  _drdy_pin  = dataReady_Pin;
  _reset_pin = reset_Pin;
  _cs_pin    = cs_Pin;
+ _clk_pin   = clk;

+ _miso_pin  = miso;

+ _mosi_pin  = mosi;



  pinMode(_drdy_pin,  INPUT);
  pinMode(_reset_pin, OUTPUT);
  pinMode(_cs_pin,    OUTPUT);

- // Start SPI

- SPI.begin(clk, miso, mosi);

- SPI.beginTransaction(SPISettings(5000000, MSBFIRST, SPI_MODE1));

+ // Keep device deselected and out of reset while bus/peripherals settle.

+ digitalWrite(_cs_pin,    HIGH);

+ digitalWrite(_reset_pin, LOW); // Necesito que inicialice en LOW para POWER-UP
SEQUENCING
```

> *[DS]* ***Datasheet p. 65:*** *"Before device power-up, all digital and analog inputs must be low."*
> ***p. 26:*** *CS debe estar HIGH cuando no hay comunicación SPI activa.*

[+] Pros	[!] Contras
CS HIGH antes de cualquier comunicación SPI	El usuario DEBE llamar `begin()` para que el chip funcione
RESET LOW en el constructor: chip en reset hasta que `begin()` lo libere	Si `begin()` nunca se llama, el ADS queda permanentemente en reset
Elimina actividad SPI prematura que corrompía el estado interno del ADS1198	—

C4 · `ADS119X.cpp` — `begin()`: secuencia tPOR + tRST corregida

Descripción: La secuencia original hacía `LOW→delay(10)→HIGH→delay(10)` y enviaba `RESET` por SPI (antes de que `SPI.begin()` se hubiera llamado). La nueva secuencia respeta los tiempos `tPOR` y `tRST` del datasheet y mueve `SPI.begin()` a este punto.

```
- boolean ADS119X::begin()
- {
-     digitalWrite(_reset_pin, LOW);
-     delay(10);
-     digitalWrite(_reset_pin, HIGH);
-     delay(10);
-     sendCommand(ADS119X_CMD_RESET);  // ❌ SPI no iniciado aún
-     delay(100);
+ boolean ADS119X::begin(){
+   //POWER-UP SEQUENCING-------------------------------------------------
+   //Todas las entradas del ADS must be LOW preferiblemente por Hardware
+
+   //1º External Clock --> No --> CLKSEL Pin=1. Ya lo está por hardware
```

```
+
+   delay(1000); // Espero tPOR a que se estabilice al energizar
+   digitalWrite(_reset_pin, HIGH); //Al inicializar estaba en LOW, preparo para tRST
+   delay(1000); // Espero 1s tras poner /RESET a 1
+   //Generamos Ancho de pulso de tiempo tRST
+   digitalWrite(_reset_pin, LOW );
+   delay(10);
+   digitalWrite(_reset_pin, HIGH );
+   //Esperamos 18tCLK antes de usar el dispositivo
+   delay(10);
+   //Ya podemos usar el dispositivo
+   //-----------------------------------------------------------------------
+
+   // Start SPI ----
+   SPI.begin(_clk_pin, _miso_pin, _mosi_pin);
+   //Teóricamente podemos llegar hasta 17Mhz
+   SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE1));
+
+   //Este command podría sobrar
+   //sendCommand(ADS119X_CMD_RESET); // Prefiero no usar SPI antes de SPI.begin
+   //delay(100);
+   //---------------
```

> *[DS]* ***Datasheet Tabla 11, p. 65:*** `tPOR ≥ 2^16 tCLK` ; `tRST ≥ 2 tCLK` . *Esperar 18 tCLK tras liberar RESET antes de usar el dispositivo.*

[+] Pros	[!] Contras
Cumple con `tPOR` y `tRST` del datasheet	~2 s de delay total en `begin()` penaliza el tiempo de arranque
`SPI.begin()` ocurre después de que el ADS completó su reset interno	Si se necesita reinicialización rápida (watchdog), el retardo puede ser problemático
Elimina el `sendCommand(RESET)` por SPI inválido	1 MHz limita throughput de configuración (irrelevante en práctica)
1 MHz de SPI da margen ante el aislador ADuM3151 (skew)	—

C5 · `ADS119X.cpp` — `begin()` : loop de 3 reintentos para validar ID

Descripción: El código original leía los registros una sola vez. Si el chip no había terminado su inicialización interna exactamente en ese instante, fallaba aunque el chip estuviera en perfecto estado. Además, el dispositivo arranca en modo RDATAC por defecto, por lo que hay que enviar SDATAC antes de leer registros.

```
- // Read All registers, save locally and make a default copy.

- syncRegData();

- for (byte adrs = 0; adrs < ADS119X_REG_SIZE; adrs++)

-     _regDataDefault[adrs] = _regData[adrs];

-

- if ((_regData[ADS119X_ADD_ID] & ADS119X_ID_7_2) == ADS119X_ID_7_2) {
```

```
-     getNumberOfChannelsFromReg();
-     ...
+ bool idOk = false;
+ // Some boards need more than one register read after reset before ID is stable.
+ for (byte attempt = 0; attempt < 3; attempt++) {
+     // Stop continuous conversion
+     sendCommand(ADS119X_CMD_SDATAC);
+     delay(10);
+
+     // Read all registers and validate ID.
+     syncRegData();
+     if ((_regData[ADS119X_ADD_ID] & ADS119X_ID_7_2) == ADS119X_ID_7_2) {
+         idOk = true;
+         break;
+     }
+     delay(10);
+ }
+
+ // If the register corresponds to an ADS119X ID
+ if (idOk) {
+     // Save locally and make a default copy.
+     for (byte adrs = 0; adrs < ADS119X_REG_SIZE; adrs++)
+         _regDataDefault[adrs] = _regData[adrs];
```

```
+
+     getNumberOfChannelsFromReg();
+     ...
```

> *[DS]* ***Datasheet p. 36:*** *"RDATAC mode is the default mode at power-up." Se debe enviar SDATAC antes de poder leer registros con RREG.*

[+] Pros	[!] Contras
Tolerancia a variación en el tiempo de estabilización del bus	Añade hasta ~60 ms adicionales en el peor caso (3 intentos fallidos)
SDATAC antes de cada lectura — necesario al arrancar en RDATAC	No distingue entre "chip ausente" y "chip lento"
`_regDataDefault` se guarda solo si el ID es válido — más correcto	—

C6 · `ADS119X.cpp` — `testSignal()`: CMD_START añadido

Descripción: `testSignal()` enviaba RDATAC sin haber enviado START. Sin START activo el chip no genera conversiones ni pulsos DRDY, por lo que el `while(!isDRDY())` habría bloqueado indefinidamente.

```
  setAllChannelMux(ADS119X_CHnSET_MUX_TEST);
+ //    Send START command if pin Start is tied low in the board
+ sendCommand(ADS119X_CMD_START);

  //    Put the Device Back in RDATAC Mode, send RDATAC
  sendCommand(ADS119X_CMD_RDATAC);
  delay(10);
```

```
  while (!isDRDY()) { }
  readChannelData();
```

> *[DS]* ***Datasheet Fig. 58, p. 66:*** `Set START = 1 // Activate Conversion` *aparece explícitamente* ***antes*** *de RDATAC en el flowchart de inicialización.*

[+] Pros	[!] Contras
Corrige bug real: sin START el `while(!isDRDY())` bloquea indefinidamente	Ninguno significativo — era un bug claro
Alinea con el flowchart del datasheet (Fig. 58)	—
`testNoise()` ya tenía CMD_START — ahora las tres funciones son consistentes	—

C7 · `ADS119X.cpp` — `testSignalDC()`: CMD_START añadido

Descripción: Mismo bug que C6, aplicado a `testSignalDC()`.

```
  setAllChannelMux(ADS119X_CHnSET_MUX_TEST);

+ //    Send START command if pin Start is tied low in the board

+ sendCommand(ADS119X_CMD_START);



  //    Put the Device Back in RDATAC Mode, send RDATAC
  sendCommand(ADS119X_CMD_RDATAC);
  delay(10);

  while (!isDRDY()) { }
  readChannelData();
```

[+] Pros	[!] Contras
Corrige mismo bug que C6 en la variante DC	Ninguno

[+] Pros	[!] Contras
Las tres funciones de test quedan simétricas	—

C8 · `main.ino` — Inicialización explícita de pines en `setup()`

Descripción: El datasheet exige que todos los pines estén en LOW antes del power-up del ADS1198. El ESP32 no garantiza el nivel de sus pines durante el boot. El bloque nuevo fuerza el estado correcto antes de llamar a `begin()`.

Además se añade `start_pin` como **variable global** junto al resto de pines, antes de la instancia `adc`:

```
  // Variables globales — declaración de pines
  byte mosi_pin = 7;
+ byte start_pin = 3;                  // ← añadido como variable global

  ADS119X adc(dataReady_Pin, reset_pin, cs_pin, clk_pin, miso_pin, mosi_pin);
```

Y su uso dentro de `setup()`:

```
  void setup() {
+   Serial.begin(115200);

+

+   // INICIO DE ADS1198-----------------------------------------------------

+   // 0. Necesario poner entradas del ADS a 0 por hardware.-----------------

+   // Estado seguro de lineas de control antes de adc.begin()

+   pinMode(cs_pin,        OUTPUT);

+   digitalWrite(cs_pin,   HIGH);   // Mantener ADS deseleccionado

+

+   pinMode(reset_pin,     OUTPUT);

+   digitalWrite(reset_pin, LOW);   // Mantener ADS en reset al arranque
```

```
+
+   pinMode(clk_pin,       OUTPUT);
+   digitalWrite(clk_pin,  LOW);
+
+   pinMode(mosi_pin,      OUTPUT);
+   digitalWrite(mosi_pin, LOW);
+
+   pinMode(start_pin,     OUTPUT);
+   digitalWrite(start_pin, LOW);
+
+   // Lineas de entrada al micro (no forzar nivel)
+   pinMode(dataReady_Pin, INPUT);
+   pinMode(miso_pin,      INPUT);
+   // Las entradas analógicas del ADC deben estar en LOW de alguna manera
+   //---------------------------------------------------------------------
+
+   // 1. Retardo inicial para "despertar"-----------------------------------

    Serial.println("Esperando estabilización de energía...");
    delay(1000); //Ya no es necesario
-   Serial.begin(115200);
-   WiFi.mode(WIFI_STA);         // ← antes estaba aquí, ahora va después de begin()
-   esp_now_init();
-   ...
```

```
+   // 2. Bucle de reintentos
+   ...adc.begin()...
+
+   // WiFi/ESP-NOW van DESPUÉS de que el ADS esté inicializado
+   WiFi.mode(WIFI_STA);
+   esp_now_init();
```

> *[DS]* ***Datasheet p. 65:*** *"Before device power-up, all digital and analog inputs must be low."*

[+] Pros	[!] Contras
Garantiza niveles definidos antes de energizar el ADS1198	Si el hardware ya tiene pull-downs correctos, el bloque es redundante (no perjudica)
CS HIGH evita transacciones SPI accidentales durante el boot	`start_pin = 3` requiere conexión hardware si se quiere usar
La radio WiFi no genera transitorios de corriente durante el power-up del ADS	Si `begin()` falla y hace `ESP.restart()`, el ESP-NOW nunca llega a inicializarse en ese ciclo (comportamiento deseado)
`Serial.begin()` movido al inicio del `setup()` — más correcto	—

C9 · `main.ino` — CMD_START explícito antes de RDATAC

Descripción: Tras la configuración de registros post-`begin()`, se añade `CMD_START` antes de `CMD_RDATAC`. Garantiza que las conversiones estén activas al entrar en `loop()`, independientemente del estado que haya dejado `testSignal()` dentro de `begin()`.

```
  adc.sendCommand(ADS119X_CMD_SDATAC);
  adc.setAllChannelGain(ADS119X_CHnSET_GAIN_12);
```

```
  adc.setAllChannelMux(ADS119X_CHnSET_MUX_NORMAL);
  adc.setDataRate(ADS119X_DRATE_500SPS);
+ adc.sendCommand(ADS119X_CMD_START);   // robustez: garantiza conversiones activas

  adc.sendCommand(ADS119X_CMD_RDATAC);
```

> *[DS]* ***Datasheet p. 35:*** *"If conversions are in progress this command has no effect." — enviar START cuando ya está activo es seguro.*

[+] Pros	[!] Contras
Garantiza conversiones activas al entrar en `loop()`	Si START ya estaba activo (pin en HIGH), el comando no tiene efecto — inofensivo
Consistente con el flowchart del datasheet (Fig. 58, p. 66)	—
Coste mínimo: un byte SPI extra	—

Conformidad con el Datasheet SBAS471C

Esta sección documenta cómo los cambios implementados se corresponden directamente con los requisitos del datasheet, concretamente con la **sección Power-Up Sequencing (p. 65)** y el **flowchart Fig. 58 (p. 66)**.

Tabla 11 — Power-Up Sequence Timing

El datasheet define tiempos mínimos obligatorios antes de poder usar el dispositivo:

Símbolo	Descripción	Mínimo	Unidad	Implementación en el código
`tPOR`	Espera tras power-up hasta aplicar reset	2^16 tCLK (~32 ms)	tCLK	`delay(1000)` en `begin()` — margen conservador de 1 s
`tRST`	Anchura mínima del pulso LOW en RESET	2 tCLK	tCLK	`delay(10)` durante el pulso LOW — ampliamente suficiente
—		18 tCLK	tCLK	`delay(10)` tras `RESET = HIGH` en `begin()`

Símbolo	Descripción	Mínimo	Unidad	Implementación en el código
	Espera tras liberar RESET antes de usar el dispositivo			

> ***Nota:*** *Con* `fCLK = 2.048 MHz`, `tPOR min = 2^16 / 2.048e6 ~ 32 ms` *y* `tRST min ~ 1 us`. *Los delays usados (1000 ms y 10 ms) son intencionadamente conservadores para absorber variaciones de la alimentación y del aislador ADuM3151.*

Fig. 58 — Flowchart de inicialización (p. 66)

El flowchart del datasheet define la secuencia completa desde el power-up hasta la captura de datos. Cada paso del diagrama se mapea con el código implementado:

Paso del flowchart	Requisito del datasheet	Donde se cumple en el codigo
Analog/Digital Power-Up	Todas las entradas en LOW antes del power-up	`main.ino` setup(): bloque `pinMode` + `digitalWrite(LOW)` antes de `begin()` **(C8)**
Set CLKSEL Pin = 1	Usar oscilador interno si no hay reloj externo	Fijado por hardware (CLKSEL conectado a DVDD)
Set PWDN=1, RESET=1, Wait 1s	Esperar `tPOR` para power-on reset y arranque del oscilador	Constructor: `RESET=LOW`. `begin()`: `delay(1000)` → `RESET=HIGH` → `delay(1000)` **(C4)**
Issue Reset Pulse, Wait 18 tCLK	Pulso LOW minimo de 2 tCLK en RESET, luego esperar 18 tCLK	`begin()`: `RESET=LOW` → `delay(10)` → `RESET=HIGH` → `delay(10)` **(C4)**
Send SDATAC	El dispositivo arranca en RDATAC — enviar SDATAC antes de leer registros	`begin()`: cada iteracion del loop de reintentos envia `CMD_SDATAC` **(C5)**
Set PDB_REFBUF=1, Wait 150 ms	Habilitar referencia interna y esperar estabilizacion	`testNoise()`: `WREG CONFIG3 0x80` (ya estaba en el codigo original)

Paso del flowchart	Requisito del datasheet	Donde se cumple en el codigo
Write Registers	Configurar data rate, canales, etc.	`main.ino` setup(): `setAllChannelGain()`, `setAllChannelMux()`, `setDataRate()`
Set START = 1	Activar conversiones antes de RDATAC	`main.ino` setup(): `sendCommand(CMD_START)` explicito **(C9)**
RDATAC	Habilitar modo de lectura continua	`main.ino` setup(): `sendCommand(CMD_RDATAC)`
Capture Data and Check Noise	Verificar ruido con entradas en corto	`begin()` : llamada a `testNoise()`
Set Test Signals, Capture Data	Verificar senal de test interna (1 mV square-wave)	`begin()` : llamada a `testSignal()` con `CMD_START` previo **(C6)**

Secuencia completa implementada

```
// main.ino - setup()
1. Todos los pines a niveles seguros          → datasheet: "all inputs must be low"

2. adc.begin() {
     delay(1000)                              → tPOR (margen 1 s)
     RESET = HIGH
     delay(1000)                              → espera estabilizacion referencia
     RESET = LOW → delay(10) → RESET = HIGH   → pulso tRST
     delay(10)                                → 18 tCLK
     SPI.begin()                              → bus SPI iniciado DESPUES del reset
     loop x3 {
       CMD_SDATAC                             → salir de RDATAC para leer registros
       syncRegData() + validar ID             → verificar comunicacion
     }
     testNoise()  → WREG CONFIG3 0x80         → referencia interna habilitada
     testSignal() → CMD_START + RDATAC        → senal de test verificada
```

```
    }

3. SDATAC + configurar registros              → data rate, gain, mux
4. CMD_START                                  → activar conversiones
5. CMD_RDATAC                                 → modo lectura continua

// loop() - lectura de datos activa
```

Resumen

#	Archivo	Cambio	Impacto
C1	`ADS119X.h`	Header guard `ADS1256_h` → `ADS119X_h`	Correctivo
C2	`ADS119X.h`	Nuevos miembros `_clk_pin`, `_miso_pin`, `_mosi_pin`	Necesario para C3/C4
C3	`ADS119X.cpp`	Constructor: `SPI.begin()` eliminado, CS=HIGH, RESET=LOW	Elimina race condition
C4	`ADS119X.cpp`	`begin()`: secuencia tPOR + tRST correcta, `SPI.begin()` movido aquí	Fix principal
C5	`ADS119X.cpp`	`begin()`: loop 3 reintentos + SDATAC previo a lectura de ID	Robustez
C6	`ADS119X.cpp`	`testSignal()`: `CMD_START` antes de `RDATAC`	Fix bug bloqueo
C7	`ADS119X.cpp`	`testSignalDC()`: `CMD_START` antes de `RDATAC`	Fix bug bloqueo
C8	`main.ino`	Inicialización explícita de pines + WiFi movido después de `begin()`	Cumple datasheet p. 65
C9	`main.ino`	`CMD_START` explícito antes de `CMD_RDATAC` al final de `setup()`	Robustez

---

Referencia principal: ADS1194/6/8 datasheet SBAS471C, secciones Power-Up Sequencing (p. 65), Setting the Device for Basic Data Capture (p. 65–66), Fig. 58 (p. 66).